Distributed Systems Assignment 3: Spark

Tundrell Group, the parent company of Tundrell Outfitters from Assignment 1, runs a Swiss web shop for books, games, garden, kitchen, music, sports, toys and tools. In Assignment 1, each new problem to be solved required new Map and Reduce code. Now Tundrell decided to replace this system with a Spark cluster. The CTO now asks you to rewrite some of the use cases to the cluster.

Before you start

The assignment repository: https://github.com/HSG-DS-HS26/BCS-DS-Assignment-3

On GitHub, select **Use this template** -> **Create a new repository** to create a private copy of the assignment repository. Do not fork the template or work in it directly.

In your private repository, open **Settings** -> **Collaborators**, add `jangrau13` and grant the account **Read** access. This allows the TA to view your code, commits and results without changing the repository. If you prefer an archive submission or do not want to grant repository access, tell the TA before the deadline and follow the archive and Teams viva process described below.

Start the cluster as described in README.md.

Task 1: word count with RDDs and Spark (1pt)

The CTO has been reading about statistical watermarks in model-generated text. She brings two texts to the weekly engineering meeting and claims that Claude's writing should leave a recognizable pattern in its word frequencies. Nobody in the room is willing to trust a visual inspection, and the MapReduce word count from Assignment 1 is now outdated. Your first Spark job will give the discussion some data.

Implementation task: implement the wordcount example using Spark and print the 20 most frequent words in each text and write the full word counts to a file.

See lecture slides "Counting Words in Spark" for guidance.

Todo:

- ☐ In `WordCount.java`, build `counts`: split the lines into lowercase words and count each word
- ☐ Write all counts to `data/output/wordcount` (printing the 20 most frequent words is supplied)
- ☐ Run `./run.sh 01_wordcount`
- ☐ Open the run in the History Server and find its jobs, stages and tasks
- ☐ Rerun with `reduceByKey(Integer::sum, 2)`, compare the number of tasks per stage, then remove the 2 again

Task 2: join with RDDs (1pt)

Management wants to open an actual physical shop, and everyone has a favorite canton/place for where the shop should be located. In order to not get into fights, data needs to decide. In theory order data and the customer data is already available, but no-one has combined the data yet. Orders contain the revenue, while customer records contain the canton.

As you might know from the database course, a join combines records from 2 datasets that share a common key. An inner join keeps a record only when its key appears in both datasets. A small example with `customer_id` as the key:

orders: order_id	customer_id	amount
1	7	30.00
2	7	12.50
3	9	80.00

customers: customer_id	canton
7	ZH
8	BE

Keyed by `customer_id`, `orders.join(customers)` returns:

customer_id	(amount, canton)
7	(30.00, ZH)
7	(12.50, ZH)

Order 3 is missing because no customer 9 exists, and customer 8 is missing because it has no orders. Customer 7 appears twice, once per order. Grouped by canton, the result is ZH with 2 orders and 42.50 CHF.

Implementation task: Implement the Spark code for the join and print the decisive report: the number of orders and the revenue in CHF per canton for 2025, sorted by canton.

Todo:

- ☐ In `Join.java`, read both files and key orders and customers by `customer_id`
- ☐ Join them, re-key by canton and add up the number of orders and the revenue per canton
- ☐ Run `./run.sh 02_join`; printing the lineage (`toDebugString`) and the table per canton is supplied
- ☐ Analyze the jobs, stages and executors in the History Server and find out how they match the code - what would happen if we didn't need the orders, just the revenue?

Task 3: PageRank with RDDs (2pt)

Management wants to improve the customer journey in the web shop and therefore needs to know which pages matter most. Today the search ranks pages only by how well their text matches the query, so a buying guide that hardly any page links to can appear above the product or category page that most of the shop links to. The search team wants a ranking signal based on the shop's own link graph, so pages referenced throughout a lot the site rise toward the top. PageRank can produce that signal, but as you might have heard in the lecture its repeated rounds expose a costly habit of MapReduce: reading intermediate data from disk again and again. Spark can keep the graph in memory while the ranks change.

Implementation task: Implement the PageRank algorithm with Spark. Run 10 iterations with a damping factor of 0.85, then print the 10 pages of the web shop with the highest PageRank and write the ranks of all pages to a file. The loop in `PageRank.java` is supplied and runs 10 iterations unless `run.sh` passes another number.

The input is a crawl of the web shop with an HTML file per page. `src/common/Html.java` is supplied: `Html.page(path)` turns a HTML file path into the page name, and `Html.links(html)` returns the links of a page to products, categories and guides. Each iteration joins `links` with `ranks`, the operation from Task 2. The guide explains caching.

Todo:

- ☐ In `PageRank.java`, build `links` with `Html.page` and `Html.links`, and cache it
- ☐ Build `ranks` (every page starts at 1.0) and write a PageRank iteration inside the loop; printing the top 10 and writing all ranks to `data/output/pagerank` is supplied
- ☐ Run `./run.sh 03_pagerank` for 10 iterations and `./run.sh 03_pagerank 3` for 3 iterations
- ☐ Compare the number of jobs and stages of both runs in the History Server
- ☐ Rerun without `.cache()` on `links`, compare Input and Shuffle Read with the cached run, then put `.cache()` back

Task 4: improving Spark code (2pt)

Ahead of Black Friday, Tundrell reviewed last year's IT performance and discovered that payments on `/checkout` had failed for 40 minutes. The operations team learned about the outage only after customers complained. Its `ServerErrors` alerting job identifies the pages with the most failures in the web shop's access logs in `data/logs`. Each run occupies the shared cluster long enough to slow the canton report and other teams' jobs, so the job runs only once per hour. The current implementation produces the correct result, but the head of operations wants it efficient enough to run every few minutes.

You bill Tundrell by the hour, so the CTO wants evidence that the optimization justifies its cost. Record measurements from before and after the change in CTO_REPORT.md.

Implementation task: Optimize the existing Spark code to print the same top 10 while sending less data through the shuffle and running fewer stages. On slower machines such as Codespaces, the optimized version also runs noticeably faster. Pages with equal error counts may appear in a different order, so tied pages near the cutoff may differ between runs. Update the comments in `ServerErrors.java` to explain how the revised operations reduce the work.

Todo:

- ☐ Run the original version with `./run.sh 04_improve_speed`
- ☐ Use the History Server to complete the question 1 table in CTO_REPORT.md: record the application duration, the number of stages, the duration and the Shuffle Write of each stage, and all shuffle operations
- ☐ Answer question 2: for each shuffle, find the line of code that causes it and the records it sends
- ☐ Optimize `ServerErrors.java`
- ☐ Confirm that the optimized version prints the same counts as the original; pages with equal counts may appear in a different order
- ☐ Run the optimized version, add its measurements to the question 1 table and answer question 3 by linking each difference to the code change that caused it

Task 5: HDFS (1pt)

For Black Friday, management plans to use the canton report from Task 2 to decide where extra stock and customer support are needed. The report is now a monthly fixture, but its input still depends on `/data`, a shared directory that works only because every container runs on the same machine. The infrastructure team wants to move the data into HDFS, where workers store replicated blocks. Before Black Friday, the CTO asks for a controlled failure test to confirm that the report still produces the correct numbers when a worker crashes.

Implementation task: Run the Task 2 canton report using data stored in HDFS, observe the extra work caused by a worker crash, verify that the output remains correct and answer questions 4 to 7 in CTO_REPORT.md.

Todo:

- ☐ Copy the join data into HDFS with the commands below
- ☐ In the NameNode UI, inspect the Datanodes page, the file sizes in `/data/join`, and the blocks and replica locations of `orders.csv`
- ☐ Run the join from HDFS and `/data`, then compare the Locality Level of the tasks that read each input
- ☐ Run the join from HDFS with a simulated worker failure and compare it with the normal HDFS run
- ☐ Answer questions 4 to 7 in CTO_REPORT.md

Copy the join data into HDFS:

```
hdfs dfs -mkdir -p /data
hdfs dfs -put -f /data/join /data/
```

The NameNode UI runs on port 9870: in Codespaces, open it from the Ports tab, and locally at http://localhost:9870.

- **Datanodes** lists the workers that store blocks, with their capacity and block count.
- **Utilities** -> **Browse the file system** -> `/data/join` lists each file with its Size, Replication and Block Size.
- Clicking `orders.csv` shows the file's blocks. The dropdown switches between the blocks, and each block lists its size and, under **Availability**, the workers that store a replica. Run the join once from HDFS and once from the shared `/data` directory. The `hdfs` argument makes `run.sh` pass `hdfs://namenode:8020/data` to the job instead of `/data`. Spark selects the storage system from the URI scheme. Arguments after the task name may appear in any order.

```
./run.sh 02_join hdfs
./run.sh 02_join
```

Run the join from HDFS again with a simulated worker failure:

```
./run.sh 02_join hdfs --problematic-system
```

The `--problematic-system` argument registers `src/common/ProblematicSystem.java`. Immediately after the first stage writes its shuffle output, the script terminates the executor and DataNode on the worker that holds the most shuffle data. The run identifies the affected worker, for example: `*** problematic system: spark-worker-2 crashes after stage 1 ***`. Compare this run with the normal `./run.sh 02_join hdfs` run:

- In the History Server, compare the total duration, stage attempts, failed tasks and error messages. Record how many tasks Spark reruns.
- On the NameNode UI's Datanodes page, monitor the failed worker and the block counts of the remaining DataNodes for about 2 minutes. The failed DataNode should disappear and then return after it restarts.

Hand in: commit and push

When Tasks 1 to 5 are done, **commit and push** to your repository. The viva reads that repository on GitHub, so work that exists only in a Codespace or on your laptop is invisible to it.

As in the other excercise, ff you do not want to work with Github, please inform the TA and schedule a private Teams call where you will get a viva in person. The same process applies if you do not want the automated viva to read your GitHub repository.

Todo:

- ☐ Tasks 1 to 5 are done, and every question in CTO_REPORT.md is answered
- ☐ **Commit and push**, or send the agreed `.zip` archive
- ☐ Tell the TA before the deadline if you want to do the viva in a Teams call instead of with the viva site

Solidify your knowledge (1pt)

Open the viva site and sign in with GitHub. Select your repository and the commit you pushed. Answer the viva's questions about your code and the contents of `CTO_REPORT.md`.

When the viva ends, **it commits its record to your repository and pushes the commit**. The commit is on GitHub but not yet in your Codespace or on your laptop. **Pull** before making more changes, or your next push will be rejected.

Todo:

- ☐ Open the viva site, select your repository and commit, and answer its questions
- ☐ **Pull** and check that `.viva/` is in your repository. In VS Code, choose **Source Control** -> **Sync Changes**, or run `git pull` in a terminal
- ☐ Attend the agreed Teams viva if you use an archive submission or withhold GitHub access from the automated viva

Submit on Canvas

The final submission is a PDF on Canvas that identifies your repository and the commit to grade. A repository tool creates the PDF and refuses to run while GitHub is missing any of your work.

1. **Commit and push** everything. If you have not pulled since the viva, pull first.
2. **Write the PDF.** In VS Code, press **Hand in** in the status bar at the bottom, or run **Terminal** -> **Run Task** -> **Hand in: write the PDF for Canvas**. From a terminal:

```
./submit.sh
```

On a Windows host without the Dev Container, run `.\submit.ps1`. If Windows refuses to run scripts, use `java submission\Submission.java`.

The tool writes `<your-username>-<repo>-submission.pdf` in the repository root and lists what it found. If it says **No PDF written**, it explains the cause and the required action, usually a commit, push or pull.

3. **Upload that PDF** to the assignment on Canvas.

If you change anything after that, **commit and push again, then write and upload a new PDF**. The PDF names a single commit, and that commit is graded.

Todo:

- ☐ Everything committed and pushed, and pulled since the viva
- ☐ **Hand in** pressed, and the PDF lists your repository and the commit you expect
- ☐ PDF uploaded to Canvas

Appendix: PageRank damping factor

PageRank models a reader who clicks through the shop at random. On every page, the reader follows a random link with probability 0.85, the damping factor, and jumps to any page of the shop with probability 0.15. A page's rank measures how often the reader lands on it. Each iteration therefore computes:

```
new rank = (1 - DAMPING) + DAMPING * sum
         =      0.15     +   0.85  * (rank / number of links, over every page that links here)
```

The jump keeps the ranks meaningful. Without it, a page with no outgoing links would absorb the rank it receives and pass none on, and pages that only link to each other would collect the rank of the whole shop. With it, every page keeps at least 0.15 and the ranks settle after a few iterations. The value 0.85 comes from the original PageRank paper by Brin and Page.

Appendix: History Server

The History Server records completed Spark applications/runs. In Codespaces, open port 18080 from the Ports tab. An application may take a few seconds to appear after it finishes. Find it by its application name, such as `join /data`; the application list also shows its total duration.

- Open **Jobs** to see the jobs started by actions such as `collect`, `take` and `saveAsTextFile`. `sortByKey` also starts a sampling job to determine the range boundaries. Click a job to view its DAG (directed acyclic graph). Each box represents a stage, while an arrow between stages marks a shuffle caused by a shuffle operation such as `reduceByKey` or `join`.
- Open **Stages** to compare the number of tasks, Input, Shuffle Read and Shuffle Write for each stage. A retried stage appears with another attempt. Click a stage to inspect its tasks, including their duration, executor, errors and Locality Level.
- Open **Executors** to see which workers ran tasks and how many tasks each worker completed.